

Quiz 1 Study Guide

AP Computer Science A • Yeshivah of Flatbush • Session 5 • 20 multiple-choice questions • 20 minutes

Part A — Markdown

Markdown is how every README and every written answer in this course is formatted. It is a **formatting** language, not a programming language — it has no logic, no variables, no decisions. It just describes how text should look.

Headings — the space is not optional

```
# Heading 1
## Heading 2
### Heading 3

##Heading 2    <-- BROKEN, prints literally
```

The number of `#` marks sets the level. You **must** put a space after them.

Bold and italic

You type	You get
<code>*text*</code>	<i>italic</i> — one asterisk each side
<code>**text**</code>	bold — two asterisks each side
<code>`text`</code>	inline code — backticks

Tables — the separator row is required

```
| Name | Score |
|-----|-----|
| Sam  | 95    |
```

The most common Markdown mistake in this class. Leave out the `| - - - | - - - |` separator row and the whole table collapses into one messy line of text. Column widths never matter — you can line them up neatly or not at all.

Code blocks

```
```java
System.out.println("Hello");
```
```

Three **backticks** (the key above Tab, not apostrophes), then the language name, then your code, then three more backticks on their own line. Naming the language turns on colour highlighting.

Line breaks — the one that surprises everyone

```
Line one
Line two    --> renders as: Line one Line two
```

```
Line one
--> renders as two paragraphs
Line two
```

Markdown **ignores a single line break** and joins the lines. To get a real new paragraph you must leave a completely blank line between them.

Study tip: open any `.md` file in your fork on GitHub, click the pencil icon, and flip between the **Edit** and **Preview** tabs. That instant feedback is the fastest way to make this stick.

Part B — Output & Escape Sequences

Reading `System.out.println` piece by piece

| Piece | What it is |
|----------------------|--|
| <code>System</code> | A class built into Java |
| <code>out</code> | An object inside <code>System</code> that represents the screen |
| <code>println</code> | The method being called — this is the part that does the work |
| <code>"Hi"</code> | The argument passed to the method |

`print` vs `println`

```
System.out.print("A");
System.out.print("B");
System.out.println("C");
```

OUTPUT:
ABC

`println` = **print line**: print the text, then move to a new line. `print` does the same thing without the new line. So A, B and C land side by side with no spaces between them — Java adds nothing you did not ask for.

Escape sequences

| Escape | Produces | Use it for |
|-----------------|----------------|--|
| <code>\n</code> | new line | Breaking one string across several lines |
| <code>\t</code> | tab | Lining up columns |
| <code>\"</code> | a double quote | Quotation marks inside a string |
| <code>\\</code> | one backslash | File paths |

Direction matters. An escape sequence starts with a **backslash** `\` — the key above Enter. A forward slash `/` does nothing. `"/n"` just prints the characters slash and n.

Printing a quotation mark

```
System.out.println("She said \"stop\".");
```

OUTPUT:

```
She said "stop".
```

Without the backslashes Java thinks the string ended at the second quotation mark and the code will not compile.

From Exercise 4 (QuoteBoard). A Windows file path needs `\\` for every single backslash you want to see: `"C:\\Users\\student"` prints `C:\Users\student`. The first backslash escapes the second.

Several lines from ONE statement

```
System.out.println("Name:\tSam\nAge:\t16");
```

OUTPUT:

```
Name:    Sam
```

```
Age:     16
```

One `println` can produce as many lines as you like, because `\n` works *inside* the string. The `\t` pushes each value out to the same column.

Part C — Variables & Calculations

Quotes or no quotes — this decides everything

```
int x = 10;  
int y = 4;
```

```
System.out.println(x - y);    // 6      <-- no quotes: Java does the MATH  
System.out.println("x - y"); // x - y  <-- quotes: Java prints the LETTERS
```

Integer division — the biggest trap in the course

```
int a = 9;  
int b = 2;  
System.out.println(a / b);    // 4    NOT 4.5, NOT 5
```

When both numbers are `int`, Java throws the decimal away. It does not round — it chops. $9/2$ is 4.5, which becomes 4. Likewise $1/2$ is 0 and $7/10$ is 0. You will see this again on every test this year.

You have already been bitten by this twice.

- **Exercise 8 (AverageCalculator)** — five `int` scores whose true average is `87.4`. If your program printed `87.0`, integer division ate the decimal before the answer was ever stored in a double.
- **DebugMe5** — a student scored 43 out of 50, which is 86%. The program reported `0.0%`, because `43 / 50` in `int` arithmetic is `0`. It compiled and ran happily. That is a **logic error**, and the compiler will never warn you about one.

Formulas you should be able to trace

```
// Area of a rectangle
int length = 8;
int width = 5;
System.out.println(length * width);           // 40

// Average of three numbers
int a = 10, b = 20, c = 30;
System.out.println((a + b + c) / 3);         // 20
```

Parentheses run first. In the average, $10 + 20 + 30 = 60$ happens before the division, giving $60 / 3 = 20$. Without the parentheses you would get something quite different.

Order of operations

| Level | Operators |
|-------|-----------------------|
| 1 | () parentheses |
| 2 | * / % — left to right |
| 3 | + - — left to right |

Mixing text and values with +

```
int apples = 3;
System.out.println("I have " + apples + " apples");
```

OUTPUT:
I have 3 apples

Anything inside quotation marks prints exactly as written. A variable outside the quotation marks is replaced by its value. Mind the spaces — they have to be inside the quotes.

Part D — Naming & Vocabulary

Three naming styles — and when each one is used

Java does not force you to use these, but every professional codebase does, and you are graded on them. More importantly, the style tells you what kind of thing you are looking at before you read another word.

| Style | Looks like | Use it for |
|---|--|--|
| camelCase
first word lowercase, every word after it capitalised | <code>favoriteNumber</code>
<code>studentScore</code>
<code>calculateTotal</code> | Variables and methods. Almost everything you have written so far. |
| PascalCase
every word capitalised, including the first | <code>HelloYou</code>
<code>BankAccount</code>
<code>String</code>
<code>Scanner</code> | Class names — and therefore file names too. |
| ALL_CAPS
every letter capital, words joined by underscores | <code>MAX_SCORE</code>
<code>PI</code> | final constants — values that must never change. |

```
public class BankAccount {           // PascalCase - it is a class
    final int MAX_WITHDRAWAL = 500;  // ALL_CAPS   - it is a constant
    int accountBalance = 0;          // camelCase  - it is a variable
}
```

This explains two things you have already run into.

- `String` has a capital S because it is a **class**, so it uses PascalCase. `int`, `double`, `boolean` and `char` are lowercase because they are built-in keywords, not classes.
- Your file had to be named `HelloYou.java` because the class inside it is `HelloYou` — PascalCase, matching exactly, capital letters included.

Legal vs. illegal names

The compiler enforces the rules in the left column. The convention in the right column is enforced by me and by every employer.

| Rule (the compiler rejects these) | Convention (it compiles, but do not) |
|---|--|
| Cannot start with a digit — <code>2ndPlace</code> | Capitalised variable — <code>String Name</code> looks like a class |
| No spaces — <code>student score</code> | Vague names — <code>x</code> , <code>tmp</code> , <code>a1</code> |
| No reserved words — <code>int class</code> | Names that do not say what they hold |

From Exercise 9 (BadVariables). Six of those lines would not compile. The seventh, `String Name = "Sarah";`, compiled perfectly and was still wrong — a capitalised variable name tells every future reader it is a class when it is not. Java is also **case sensitive**: `score`, `Score` and `SCORE` are three different variables.

The data types

| Type | Holds | Example | Quotes? |
|---------|----------------------|----------------------------------|--------------------------------|
| int | Whole numbers | <code>int age = 16;</code> | none |
| double | Decimal numbers | <code>double gpa = 3.75;</code> | none |
| boolean | true or false only | <code>boolean ok = true;</code> | none |
| char | One character | <code>char grade = 'A';</code> | single <code>' '</code> |
| String | Text of any length | <code>String w = "Hello";</code> | double <code>" "</code> |

Watch these two.

- `int price = 3.75;` will not compile — an int cannot hold a decimal.
- `char word = "Hello";` will not compile either — double quotes make a String, and a char holds exactly one character in single quotes.

Choosing a type — the two that fool everyone

From the Type Detective exercise. Both of these look like numbers and neither one is.

| Value | Type | Why not int |
|--|--------|---|
| A phone number | String | You never do arithmetic on it, and an int drops a leading zero |
| A student ID like <code>0074512</code> | String | <code>0074512</code> becomes <code>74512</code> — the leading zeros are gone for good |

The test is simple: **would you ever add, subtract or average these values?** If not, it is text, however many digits it has.

public vs private

| Keyword | Means |
|---------|---|
| public | Any other class in the program can reach it |
| private | Only code inside the same class can reach it |

You have already been writing `public class` and `public static void main` all along. `private` is how you protect a class's own data from being changed by accident from outside — we build classes of our own in Unit 5.

Do not confuse them. `private` controls *who can reach* a variable. It has nothing to do with whether the value can change — that is `final`.

Self-check — can you do all of these?

- ☐ Write a Heading 2 in Markdown, with the space in the right place
- ☐ Name the one row a Markdown table cannot render without
- ☐ Explain why two lines typed one under the other become a single paragraph
- ☐ Say what `print` does that `println` does not
- ☐ Point at the method in `System.out.println("Hi")`
- ☐ Write a statement that prints a double quotation mark
- ☐ Produce two tab-aligned lines from a single `println`

- ☐ Say what `\n`, `\t`, `\"` and `\\` each do, and which slash they use
- ☐ Predict the output of `9 / 2` when both values are `int`
- ☐ Compute the area of a rectangle and the average of three numbers by hand
- ☐ Explain the difference between `println(x - y)` and `println("x - y")`
- ☐ Choose between `int`, `double`, `String` and `char` for a given value
- ☐ Say what `private` means, and what it does not mean

The single best way to prepare: reopen your Session 3–5 exercises in a codespace, cover the output with your hand, and predict what each program prints before you run it. That is exactly what the quiz asks you to do.